Chapter 1 - Neurons and Layers

An artificial neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function. Stacking neurons into layers and layers into networks is what gives deep models their capacity.

Without a non-linear activation, any stack of linear layers collapses into a single linear transformation, no matter how deep it is.

Chapter 2 - Activation Functions

ReLU outputs the input when positive and zero otherwise. It is cheap to compute and does not saturate for positive inputs, which is why it largely replaced sigmoid and tanh in hidden layers.

A known failure mode of ReLU is the dying unit: if a neuron's input stays negative, its gradient is always zero and it stops learning. Leaky ReLU addresses this by allowing a small negative slope.

Chapter 3 - Loss Functions

Cross-entropy loss is the standard choice for classification, while mean squared error is standard for regression. Matching the loss to the task matters more than small architectural choices.

For imbalanced classification, a weighted loss that increases the cost of errors on the minority class often works better than resampling the data.

Chapter 4 - Optimisers

Stochastic gradient descent updates parameters using the gradient computed on a mini-batch rather than the whole dataset, trading some gradient accuracy for far more updates per unit of time.

Adam keeps running averages of both the gradient and its square, which adapts the effective step size per parameter and usually converges faster than plain stochastic gradient descent.

Chapter 5 - Initialisation

Initialising all weights to the same value makes every neuron in a layer compute the same thing and receive the same gradient, so the layer never differentiates. Random initialisation breaks this symmetry.

Xavier initialisation scales the initial weights by the number of input and output connections, keeping the variance of activations roughly constant across layers.

Chapter 6 - Regularisation in Deep Networks

Dropout randomly disables a fraction of units during training, preventing the network from relying too heavily on any single unit. At inference time all units are active and their outputs are scaled accordingly.

Weight decay adds a penalty on the squared magnitude of the weights, which for plain stochastic gradient descent is equivalent to L2 regularisation.

Chapter 7 - Normalisation Inside Networks

Batch normalisation standardises the activations of a layer using statistics computed across the current mini-batch, which stabilises training and allows larger learning rates. Its weakness is that it behaves differently at training and inference time and degrades with very small batches.

Layer Normalization instead standardises across the features of a single example rather than across the batch, so it behaves identically at training and inference time and is unaffected by batch size. That property is why it is the normalisation used inside Transformer blocks.

Chapter 8 - Practical Advice

Overfit a tiny subset of the data first. If the model cannot reach near zero loss on ten examples, there is a bug in the model or the training loop, not a shortage of data.

Change one thing at a time when tuning. Simultaneous changes make it impossible to attribute an improvement to any particular decision.